Safe to Drop: an Exact Criterion for Quantum Temporaries

ANONYMOUS AUTHOR(S)

We study when a quantum compiler can safely drop a temporary register so that reusing it does not affect the computation. Several existing systems use sufficient conditions based on how the temporary was produced or require it to be restored to its allocation state as in uncomputation. We give an exact semantic criterion in a typed language, characterized by contextual equivalence of the program prefix, with every register exposed, to a witness that prepares the temporary independently of the remaining computation. Without measurement, a temporary is safe to drop and reuse exactly when it ends in one fixed pure state for every input. One unitary on the temporary then restores its allocation state without changing the remaining registers. With measurement, the comparison is between individual records and includes their probability weights.

1 The problem

We want to be able to reuse temporary registers once their lifetimes end, as we do classically by discarding their values. Unitary quantum gates are reversible, so we usually restore a temporary by uncomputation, running the operations that produced it backwards [11, §1]. Several systems address this with automatic uncomputation, checks for safe disposal, restoration requirements, or ownership and borrowing disciplines [1–3, 5, 8, 11–15].

The main nuance is that a temporary may go out of scope while still entangled with other registers. Their joint state then cannot be split into separate states. Uncomputation can undo these correlations, but may require operations on the other registers as well [11, §3, Fig. 2]. We ask when one unitary gate on the temporary alone can restore its allocation state at every input and leave the rest intact, and call such a drop *safe*. Each program below uses qubits of type `[2]`. It allocates t in $|0\rangle$ and drops its name while keeping v. Dropping applies no physical operation. Both safe and unsafe cases are well typed. Purple marks lines added to program (a).

Example 1.1 (the running examples).

(a) *superpose, then drop*

```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;



  drop t;
  return v }
```

(b) *superpose, then read*

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  CX t v;


  drop t;
  return v }
```

(c) *copy, measure, one joint gate*

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v;  CX v t;
  p = lift[{0}] t;
  V t v;
  drop t;
  return v }
```

In program (a), the Hadamard gate H puts t in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. This is the same state whatever v held, and another H returns t to $|0\rangle$ without touching v. Program (b) adds `CX t v`, which flips v when t is 1. At input $|0\rangle_v$, this read entangles t and v. With t hidden, v is an equal mixture of $|0\rangle$ and $|1\rangle$. In both programs, t's lifetime ends at the drop, and

neither uses t afterwards. Checking those lifetimes alone would accept both, though only program (a) is safe. Program (c)'s measurement and gate V follow in section 3.

2 Separating the temporary from the computation

For the safety test, we will call one occurrence of `drop` a *site*, and the tuple of registers it names its *block*. Here each block is just t. A joint pure state of n qubits is a unit vector with 2^n complex coordinates, one per bit string. For programs (a) and (b), let K denote the prefix's state map just before the drop. It maps an input x in v to the joint state Kx of t and v. Put t first in each tensor product. Program (a) gives

$$x \xrightarrow{\texttt{new t @ |0>}} |0\rangle \otimes x \xrightarrow{\texttt{H t}} \frac{|0\rangle \otimes x + |1\rangle \otimes x}{\sqrt{2}} = |+\rangle \otimes x.$$

Similarly, program (b) has the same first two steps, followed by

$$|+\rangle \otimes x \xrightarrow{\texttt{CX t v}} \frac{|0\rangle \otimes x + |1\rangle \otimes Xx}{\sqrt{2}}.$$

Program (a) leaves v alone for both temporary labels. Program (b) leaves v alone for label 0 and applies the bit flip X for label 1.

$$K_{(a)} = \frac{1}{\sqrt{2}} \overbrace{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}}^{H \otimes I} \cdot \overbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}^{|0\rangle \otimes I} = \overbrace{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}^{K_{(a)}} \qquad \overbrace{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}}^{K_{(b)} = \mathrm{CX}\, K_{(a)}}$$

$$K_0 = K_1 = I/\sqrt{2} \qquad K_0 = I/\sqrt{2}, \quad K_1 = X/\sqrt{2}$$

Fig. 1. The pre-drop maps, in row order 00, 01, 10, 11. Blue rows have temporary label 0, gold rows label 1. The slices are equal in program (a) but not proportional in program (b).

Think of K as a `switch` on the basis labels of a temporary of dimension d. Branch i acts on the other registers through the *slice* K_i, formed by the rows labelled i (fig. 1). Pairing its result with the temporary's basis state $|i\rangle$ gives $\boldsymbol{Kx = \sum_i |i\rangle \otimes K_i x}$.

A classical switch selects one branch. The quantum sum assembles all slices into Kx, the joint state before the drop, keeping their amplitudes and phases. The drop makes these labels unavailable to the program. A gate on the temporary can restore it exactly when every slice acts the same way up to a scalar. That common action, which we call W, describes what the prefix has done to the remaining registers.

Theorem 2.1 (the criterion, without measurement). *Let K be the joint state map just before a temporary-only drop in a well-typed measurement-free program, with the dropped block first. The drop is safe exactly when there are*

- *a fixed unit vector g on the block, and*
- *a linear map W to all other registers, including those already dropped,*

such that $\forall x, \quad Kx = g \otimes Wx$.

The map W then preserves norms and inner products, so it is an isometry.

We call the fixed state g a *certificate* of safety, with coordinates g_i in $K_i = g_i W$. For example, program (a) has $g = |+\rangle$ and $W = I$. The left of fig. 2 shows this separation. A safe drop is one gate away from a fresh register. Suppose the temporary was allocated in ψ and has a fixed state g before the drop. Its overall phase does not change the state. Now consider the overlap $\langle \psi, g \rangle$.

- If the overlap is nonzero, choose the phase of g to make that overlap real and positive.
- If the overlap is zero, any phase works, so choose one that makes the first nonzero coordinate of g positive and real.

Call the chosen vector $\tilde{g}$. Absorb the inverse phase into W, keeping the same name, so the joint map stays $K = \tilde{g} \otimes W$.

If $\tilde{g} = \psi$, no gate is needed. Otherwise, the Householder reflection through the hyperplane orthogonal to $\psi - \tilde{g}$ exchanges the two states [7, p. 340, Eqs. (3)–(4)] (right of fig. 2). Applied to the temporary alone, it returns it to ψ at every input and leaves the rest unchanged. Note that this is like Grover's diffusion step that uses the same reflection form with the opposite overall sign [6, §4]. By one gate we mean one unitary on the whole block. Its hardware implementation may take a circuit. For program (a), this reflection is H. Deutsch–Jozsa uses one query to distinguish a black-box bit function promised to be constant or to return 0 on exactly half its inputs. Its temporary ends in $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ whatever that function is. The temporary is safe though not in $|0\rangle$.

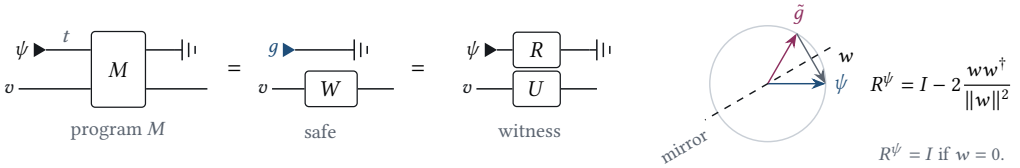


Fig. 2. Left, a witness prepares g with R and implements W with U. Triangles prepare states and ground symbols mark drops. Fresh registers for U are omitted. Right, the reflection exchanges $\tilde{g}$ and ψ. Its complex-state formula uses $w = \psi - \tilde{g}$ and conjugate transpose $\dagger$.

We also consider borrowed or dirty registers, which we call *loans*. A loan can arrive in an unknown state, possibly entangled with the caller. We must return that state with its entanglement intact. Passing a tuple to `drop` groups the registers to be returned together into one block. In `mixed` below, `drop (l,t)` groups the loan l and temporary t. The program is named for this mixed block and returns `()` with no output registers.

Unlike a temporary-only block, a block of loans and temporaries need not end in one fixed state. Its state can depend on the unknown loan input. Following the same intuition as before, returning the loan is safe when the action on that input separates from the computation on the other inputs. Without measurement, safety holds exactly when the map factors as $B \otimes W$, where the isometry B encodes the loans in the block and W acts on the other inputs. One unitary on the block can therefore undo the encoding and restore the temporaries beside the unchanged loans (fig. 3). For a loan-only block, the unitary B has return gate $B^\dagger$.

```
prog mixed (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```

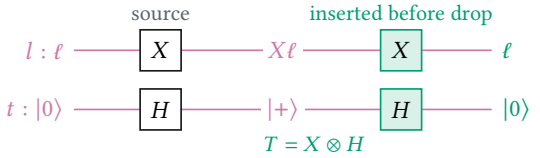


Fig. 3. The inserted $T = X \otimes H$, with the loan first, returns every loan input ℓ and restores t to $|0\rangle$. The source drop applies no gate.

3 One measurement record at a time

Our `lift` follows the control design of *dynamic lifting* [4]. It measures a register and binds the outcome as a classical parameter, which only `if` guards can read. Dynamic lifting makes measured values available during circuit generation. Our language has no separate circuit-generation stage. For us, the measurement keeps one group of coordinates and reports which group survived. For a normalized state, the outcome's probability is the squared norm of the surviving part.

In program (c), `lift[{0}]` records 1 for $t = 0$, 0 otherwise. Either outcome, when it occurs, leaves a product state at the lift. Choose V to keep $|00\rangle$ unchanged and send $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$. The outputs are orthogonal unit vectors, so this action extends to a unitary. The same program is then safe on one record and unsafe on the other, as fig. 4 shows for input $x = \alpha|0\rangle + \beta|1\rangle$.

A record o lists the outcomes before the drop. For a block of temporaries, safety on o holds exactly when the joint map again separates into one fixed block state and its own remaining action W_o. The unnormalized vector $W_o x$ keeps both the computation that survived and the weight it survived with. For a unit input x, the record's survival probability is $\|W_o x\|^2$. For example, on record 1, $W_1 x = (\alpha + \beta)|0\rangle/\sqrt{2}$. Normalizing a nonzero $W_1 x$ gives v's state $|0\rangle$, but hides how likely that record was. Input $|+\rangle$ reaches it with probability 1, whereas input $|0\rangle$ reaches it with probability $1/2$. The unnormalized map preserves these probabilities when comparing programs. A record that no input reaches has zero map and is safe.

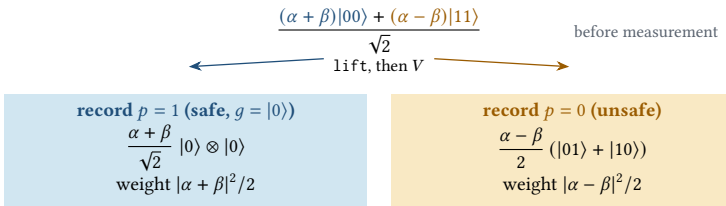


Fig. 4. In program (c), blue fixes the temporary in $|0\rangle$. Gold leaves an entangled pair when its weight is nonzero. Both records have weight $1/2$ on either basis input, but only blue is safe.

For these temporary blocks, a witness prepares the block in ψ and applies a unitary R, leaving it in $g = R\psi$ independently of the input. On the remaining registers, it allocates the other temporaries and a fresh qubit a in $|0\rangle$, then applies a suitable unitary U to everything except the block. It measures a with `lift[{0}]` and keeps the outcome in which a is $|0\rangle$, so the lift records 1. Omitting this fixed factor leaves the unnormalized map $g \otimes W_o$. The extra

qubit lets the witness realize W_o, which may reduce the norm, using unitary gates and one measurement. Without measurement, W preserves norms and needs no extra qubit. These witnesses and the contexts below may use arbitrary unitary gates. Figure 5 gives the witness for program (c).

```
prog witness (v : [2]) : [2] * [2] {
  new t : [2] @ |0>; new a : [2] @ |0>;
  H v; CX v a;
  p = lift[{0}] a; drop a; return (t,v) }
```

Keep record $p = 1$, where a is in $|0\rangle$. After omitting that fixed factor, the joint output is $|0\rangle_t \otimes W_1 x$. Here $R = I$, and U is `H v; CX v a`.

Fig. 5. A witness for program (c)'s safe record 1. The other outcome is auxiliary.

For a program M with input context Ω, we write $\sigma_s^o : [[\Omega]] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ for the unnormalized map just before drop site s on some record o, the record-specific version of K. Here $[[\Omega]]$ is the input space, $\mathcal{H}_v$ the block we intend to drop, and $\mathcal{H}_{\text{rest}}$ all other registers, including previously dropped ones. The prefix $M|_s$ stops before s, with those drops deleted. A closed context supplies inputs and tests all of the prefix's registers, block included, comparing weights without normalizing away the record probability.

Theorem 3.1 (the criterion, one record at a time). *Let M be well typed with input context Ω, and let s drop a block of temporaries allocated in state ψ. On a measurement record o whose path includes s, the following four statements are equivalent.*

(1) s is safe on o.

(2) There are a unit vector $g \in \mathcal{H}_v$ and a linear contraction $W_o : [[\Omega]] \to \mathcal{H}_{\text{rest}}$ such that:

$$\forall x, \quad \sigma_s^o x = g \otimes W_o x, \quad \|W_o x\| \le \|x\|.$$

(3) Some witness built as above has kept-record map σ_s^o, after omitting the fixed $|0\rangle_a$ factor.

(4) There is a witness as in (3) whose kept-record map, after omitting the fixed $|0\rangle_a$ factor, agrees with $M|_s$ on o in every closed context. Measurement-free contexts suffice.

4 Scope, implementation and open questions

We implemented a typed first-order language with finite-dimensional registers and classical conditionals. We prove its type safety. We also show that reusing restored safe temporaries preserves records, their weights and the reduced output state. For qubit programs over X, `CX` and Toffoli with basis-state allocations, deciding safety of temporary-only drops is coNP-complete in program-text size. This holds for one site on one record and for all sites on all records. Hardness holds even without measurement and with a single drop.

At a tuple drop, we test all the named registers together as one block. For temporaries, we test each record for one fixed pure block state that works at every input. The registers in the block can still be entangled with each other. For example, a fixed Bell pair $(|00\rangle + |11\rangle)/\sqrt{2}$ is safe as one block because its joint state is fixed and pure. Testing the registers separately gives a different answer. Each half of the Bell pair has density matrix $I/2$, the equal mixture of $|0\rangle$ and $|1\rangle$, and remains entangled with the other. It is therefore unsafe to drop each half separately. Without measurement, separate safety is equivalent to safety of one tuple drop with a product certificate, placed at the end after deleting the individual drops. A fixed mixed state does not suffice.

Since reuse must preserve observable behavior, we might hope to decide safety from the program's surviving outputs and the probabilities of its measurement records. But the program's summed output channel and record weights do not determine safety. The channel sums visible subchannels over records, tracing out hidden registers and losing their correlations with the output. Knowing the record weights does not recover those correlations. The table gives two programs' record outputs at ρ, with opposite verdicts on every record. Our test uses each record's joint map.

	$p = 1$	$p = 0$
safe	$\rho/2$	$Z\rho Z/2$
unsafe	$D(\rho)/2$	$D(\rho)/2$

sum: $D(\rho) = \mathrm{diag}(\rho)$
record weights: $1/2$

Our Haskell implementation includes a matrix-based checker for a one-temporary catalogue and an independent state-vector checker for blocks, records and loans. Both representations grow exponentially with the qubit count. The OpenQASM 3 emitter covers power-of-two dimensions, basis-state allocations, and conditionals whose branches vary only gates. It supports H, X, Z, S, T, CNOT, Toffoli, SWAP, Grover diffusion, and permutation, diagonal and one-qubit literals. It computes temporary verdicts and return gates before execution, selecting them by the outcomes observed before the drop. It emits restoring circuits for safe temporary-only blocks. For loans, it numerically recognizes factors proportional to the identity, a one-qubit gate or a basis permutation. It also restores mixed blocks when such a factor separates from a fixed temporary state.

A safe site admits a return unitary, but practical reuse still requires the following work.

- Reuse restored temporaries' physical qubits. The current emitter assigns fresh wires to later allocations. A reuse allocator would track which temporary wires are available on each record and prepare each reused wire in its next allocation state, after the required return gate has run.
- Extend loan restoration. The emitter rejects loan drops that follow a measurement or a conditional. Other unsupported loan blocks remain allocated and unreturned. General return circuits must undo the encoding even for loans entangled with the caller.
- Control numerical error. Gate parameters are numerical, whereas the theorems assume ideal unitaries. Simulation tests check selected records within a tolerance; a proved error bound would give a guarantee beyond those tests.

In the literature, we find several approaches directly relevant to our criterion.

- Silq, Unqomp and ReQWIRE provide sufficient syntactic conditions [1, 11, 12].
- Q# release, Qutes reclamation and fractional types for classical reversible programs require restoration [2, 3, 10]. Quipper's assertive termination requires a specified basis state [5]. Q# and Quipper leave these conditions to the programmer.
- Twist combines static analysis with a runtime separability check [15], Guppy tracks ownership and borrowing [8], and Bona schedules borrows assumed safe [14].
- Liu, Zhou and Meng ask whether uncomputation can restore the ancillas while leaving the working registers with the sum of the slice outputs. This requires the slice sum to be unitary, which is their functionality test [9, Defs. 3.1–3.2]. Program (a) is safe, but $\sum_i K_i = \sqrt{2}I$ fails that test because it scales norms by $\sqrt{2}$. Their uncomputation problem is coNP-complete for X and Toffoli circuits [9, Cor. 3.1].

The present language has no loops or recursion. How should a type system certify safe re-entry into temporary scopes in loops and quantum recursion, including quantum guards' scopes? When is unitary restoration preferable to measurement and reset?

References

[1] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007

[2] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[3] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph]

[4] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL (2023). doi:10.1145/3571204

[5] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *ACM SIGPLAN Notices (PLDI)* 48, 6 (2013), 333–342. doi:10.1145/2499370.2462177

[6] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[7] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[8] Mark Koch, Agustín Borgna, Craig Roy, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. Imperative Quantum Programming with Ownership and Borrowing in Guppy. arXiv:2510.13082 [cs.PL] Presented at PLanQC 2025.

[9] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. arXiv:2608.09578 [cs.PL]

[10] Microsoft. 2024. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[11] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[12] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[13] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL2018)*. Article 7, 10 pages. doi:10.1145/3183895.3183901

[14] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL]

[15] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691